



Pontificia Universidad Católica de Chile
IA Lab



IA Lab

Transformers

Felipe del Río

Computer Science Department, PUC

Motivation

Motivation

When modeling sequence data, RNNs are a natural choice to model a particular task.

But...

- Doesn't have an explicit mechanism to model hierarchy.
- hidden state $\rightarrow$ bottleneck during training
- Requires various steps to relate far-away tokens.

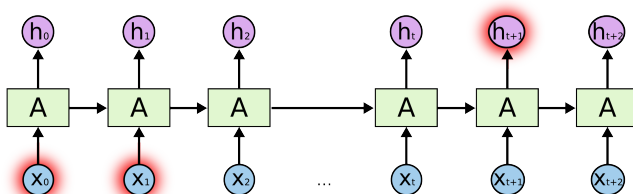


Image from <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

Table of Contents

- 1 Motivation
- 2 Transformer
- 3 Short review of seq2seq models
- 4 Transformer Architecture
- 5 Embeddings
- 6 Attention
- 7 Transformer for Multi-modal Learning
- 8 Final words
- 9 Questions?
- 10 Appendix
- 11 Transformers for NLP: BERT
- 12 CLIP
- 13 Relation Networks

Transformer

Attention Is All You Need

Ashish Vaswani*

Google Brain

avaswani@google.com

Noam Shazeer*

Google Brain

noam@google.com

Niki Parmar*

Google Research

nikip@google.com

Jakob Uszkoreit*

Google Research

usz@google.com

Llion Jones*

Google Research

llion@google.com

Aidan N. Gomez* †

University of Toronto

aidan@cs.toronto.edu

Łukasz Kaiser*

Google Brain

lukaszkaizer@google.com

Illia Polosukhin* †

illia.polosukhin@gmail.com

A paper released in 2017 by people from Google Brain & Google Research.

Seq2seq model based on **self-attention**.

Revolutionize NLP and the whole field.

Key Ingredients:

- 1 Self-attention
- 2 Multi-head Attention
- 3 Hierarchical

Short review of seq2seq models

Seq2seq Models

First, let's remember how seq2seq models work.

Transformers were proposed for machine translation.

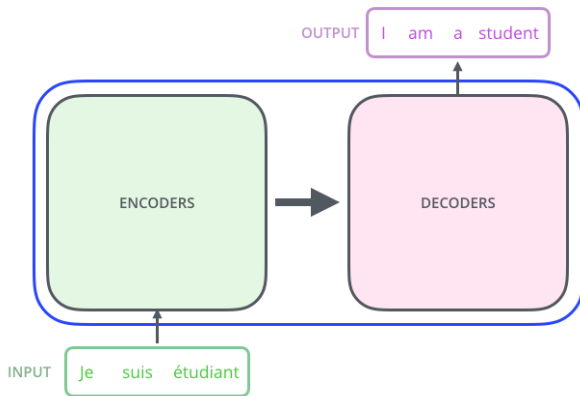
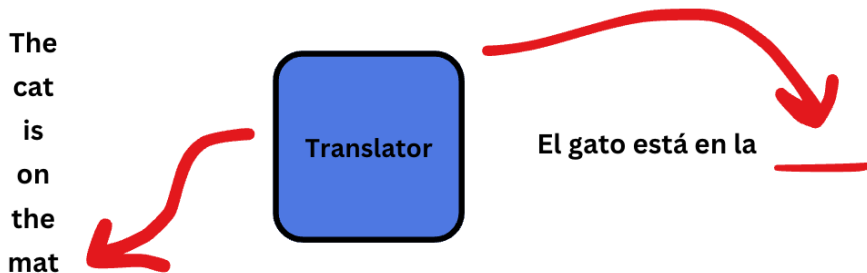


Figure: <https://jalammar.github.io/illustrated-transformer/>

Seq2seq Models

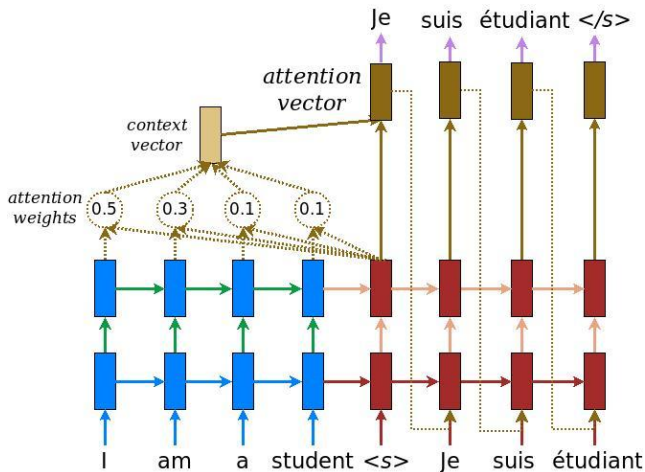
First, let's remember how seq2seq models work.

Transformers were proposed for machine translation.



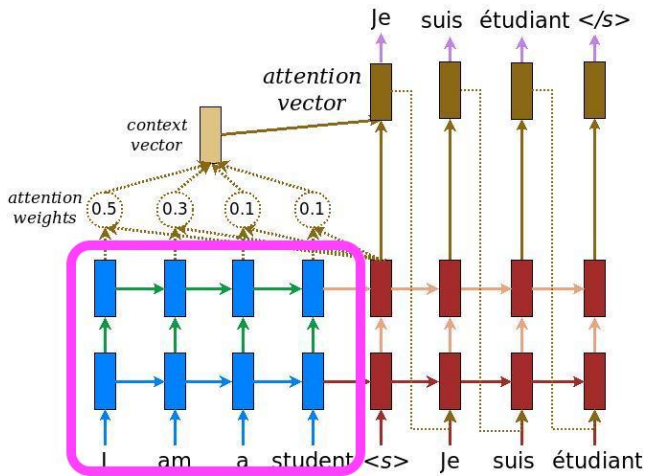
Seq2seq with attention

Review seq2seq with attention.



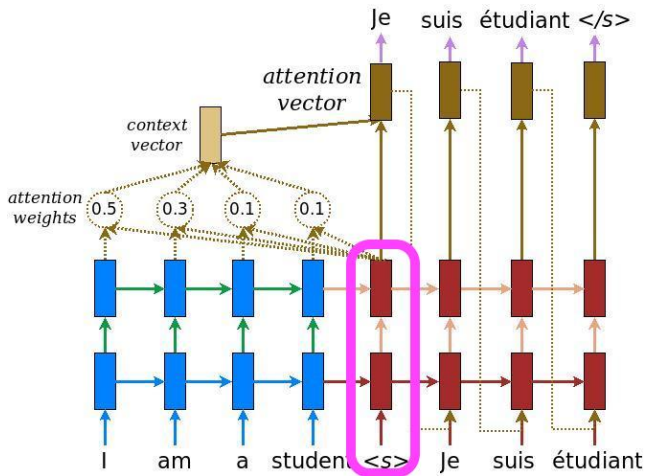
Model review: Seq2seq with Attention

Review seq2seq with attention.



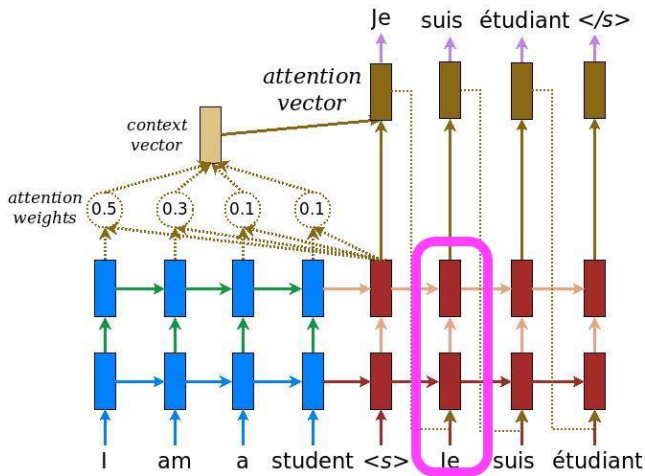
Model review: Seq2seq with Attention

Review seq2seq with attention.



Model review: Seq2seq with Attention

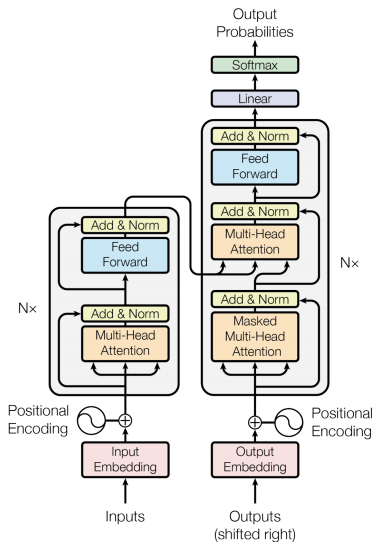
Review seq2seq with attention.



Transformer Architecture

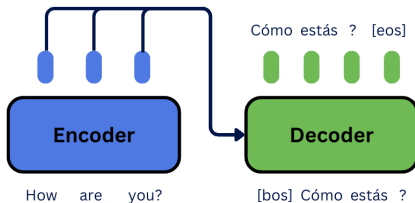
The Transformer

Let's understand its inner workings module by module.



Embeddings

Embedding Layer



Vocabulary

index:	word:
0	aardvark
1	able
...	...
2409	black
2410	bling
...	...
3202	candid
3203	cast
3204	cat
...	...
5281	is
5282	island
...	...
8676	the
8677	thing
...	...
9999	zombie

10,000
words
with
indices

Embedding Layer: From words to vectors

Function: Code input words to vectors.

Why?

- **Flexibility:** Learn the representation.
- **Utility:** Process in NNs.
- **Semantic:** Semantic space.

Parameters:

- Vocabulary size.
- Vector size.

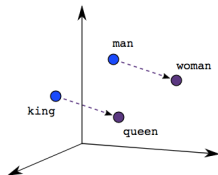
A 4-dimensional embedding

cat =>	1.2	-0.1	4.3	3.2
mat =>	0.4	2.5	-0.9	0.5
on =>	2.1	0.3	0.1	0.4

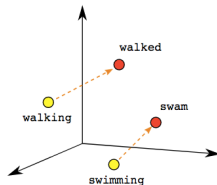
...

...

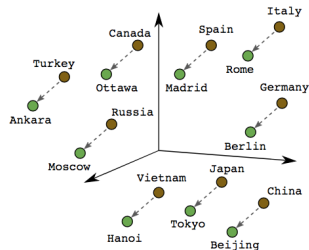
Embedding Layer: Semantic spaces



Male-Female



Verb Tense



Country-Capital

Directions and distances in space acquire semantic meaning.

Attention

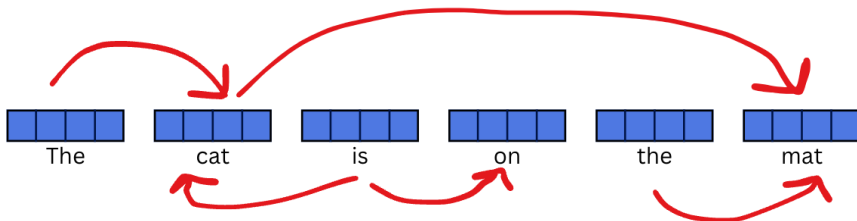
The trophy cannot fit in the suitcase because it is too big

In language, words in sentences establish relationships with themselves to compose new meanings.

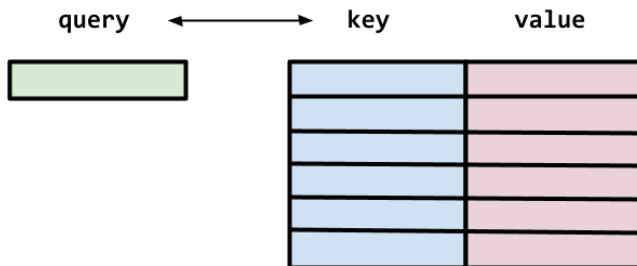


In language, words in sentences establish relationships with themselves to compose new meanings.

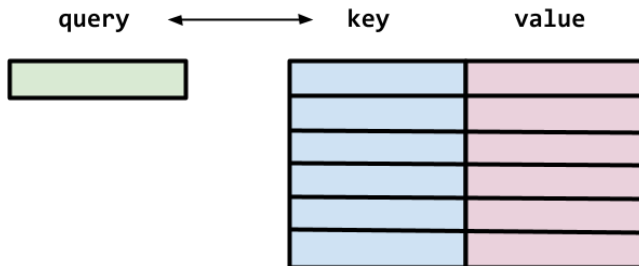
Problem: Meaning is composed between words present in a sentence.



$$\textit{Attention}(\textit{query}, \textit{key}, \textit{value})$$

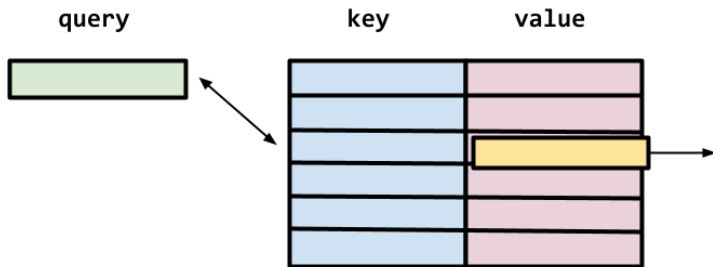


```
memory = {  
    key_0: value_0 ,  
    key_1: value_1 ,  
    key_2: value_2 ,  
    ...  
}
```



Attention Intuition

```
memory = {  
    key_0: value_0 ,  
    key_1: value_1 ,  
    key_2: value_2 ,  
    ...  
}  
memory[query] # returns value
```



The cat is on the mat

e_0 e_1 e_2 e_3 e_4 e_5

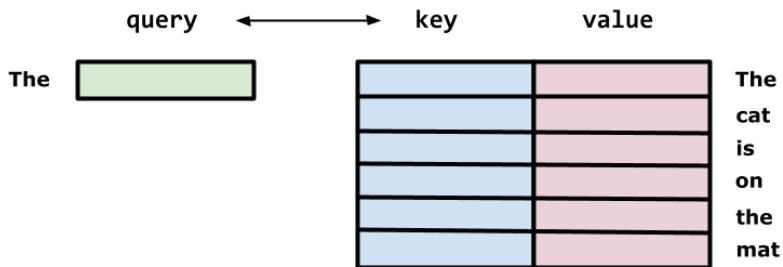
The cat is on the mat

q_0 q_1 q_2 q_3 q_4 q_5

k_0 k_1 k_2 k_3 k_4 k_5

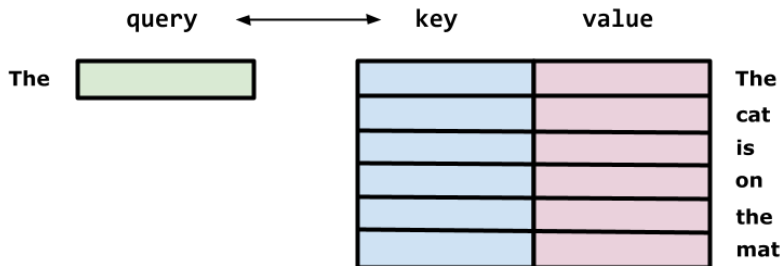
v_0 v_1 v_2 v_3 v_4 v_5

$$\textit{Attention}(Q, K, V)$$

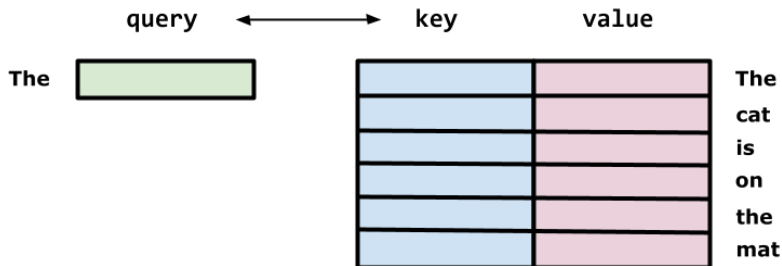


Self-Attention

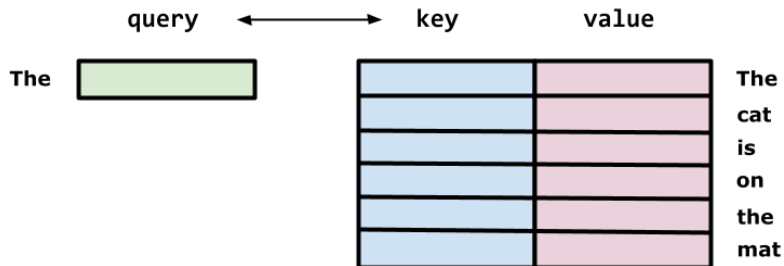
```
memory = {  
    'The': 'The ',  
    'cat': 'cat ',  
    'is': 'is ',  
    ...  
}
```



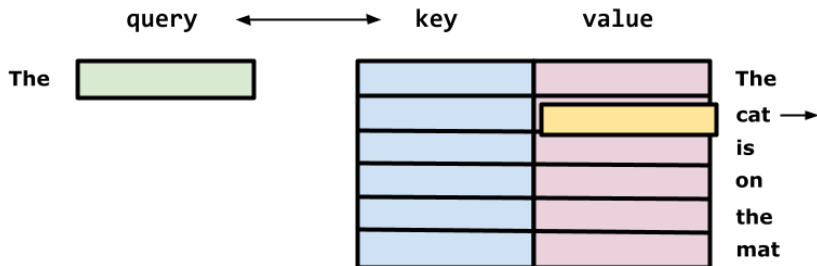
```
memory = {  
    k( 'The' ): v( 'The' ),  
    k( 'cat' ): v( 'cat' ),  
    k( 'is' ): v( 'is' ),  
    ...  
}
```



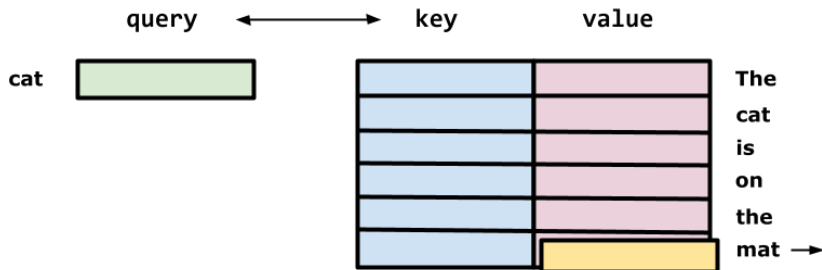
memory[q('The')]



`memory[q('The')] # returns v('cat')`

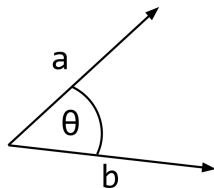


`memory[q('cat')]` *# returns* `v('mat')`



Recap: Dot Product

Geometric Interpretation:



$$\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}| |\mathbf{b}| \cos \theta$$

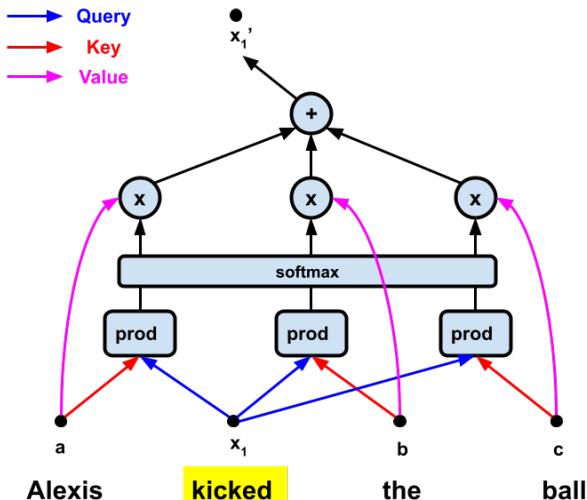
Example:

$$\begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix} \cdot \begin{bmatrix} 5 \\ 1 \\ 7 \end{bmatrix} = 2 \cdot 5 + 3 \cdot 1 + 4 \cdot 7 \\ = 41$$

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$

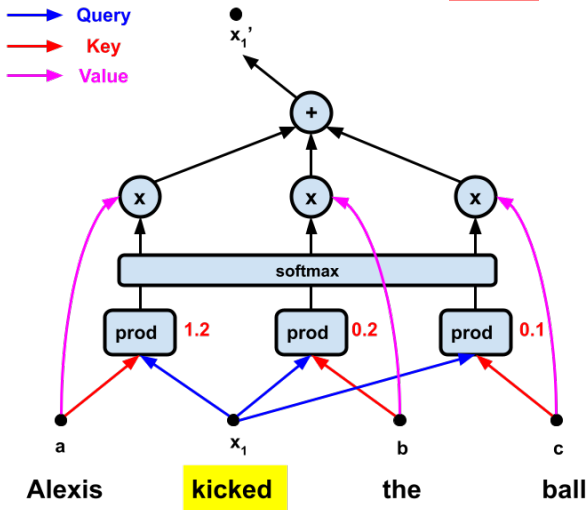
Self-Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



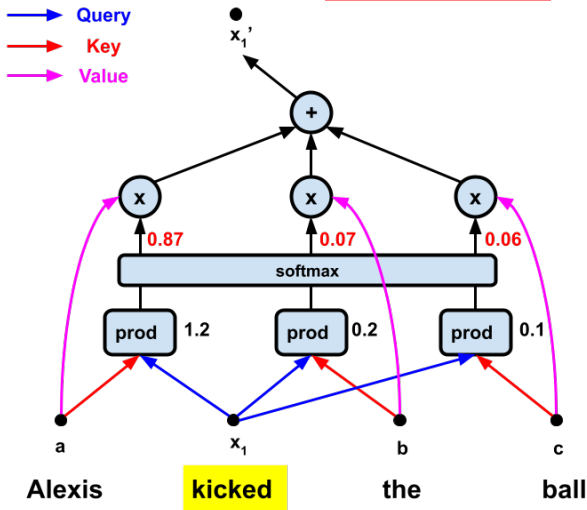
Self-Attention

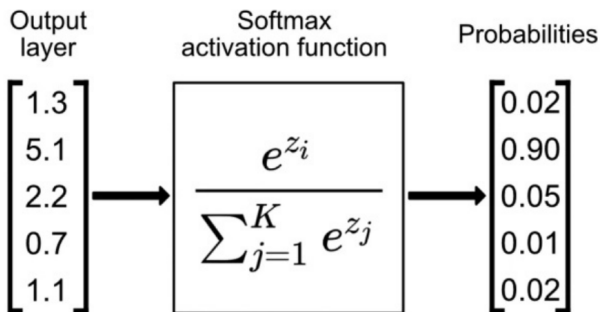
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



Self-Attention

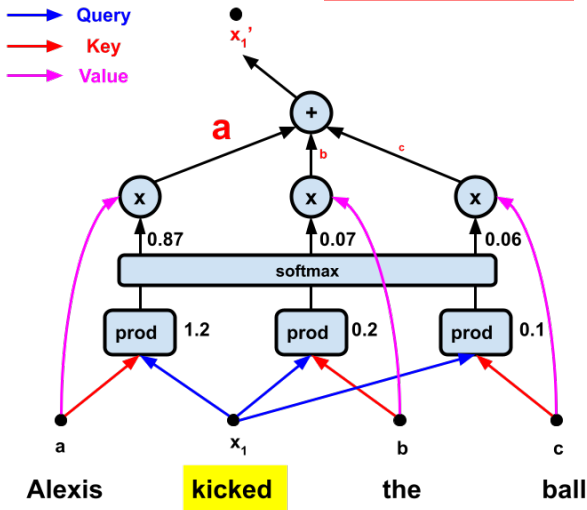
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$





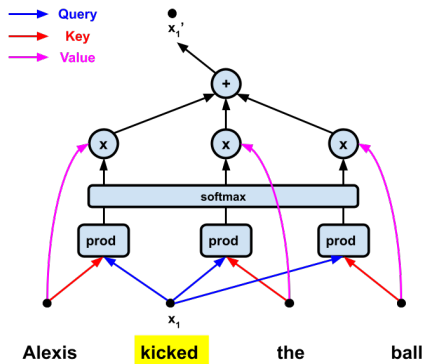
Self-Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

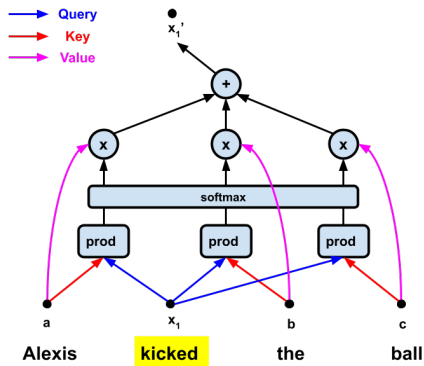


Self-Attention

Re-express yourself in terms of a weighted combination of your neighborhood.



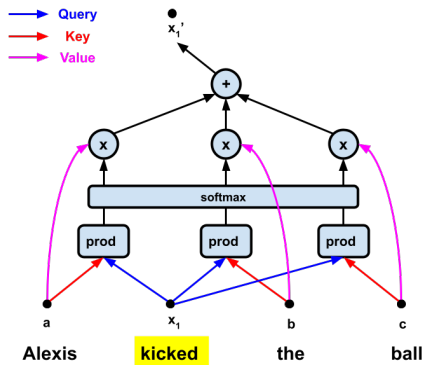
Contextualize every token representation.



Why Self-Attention?

Advantages of self-attention

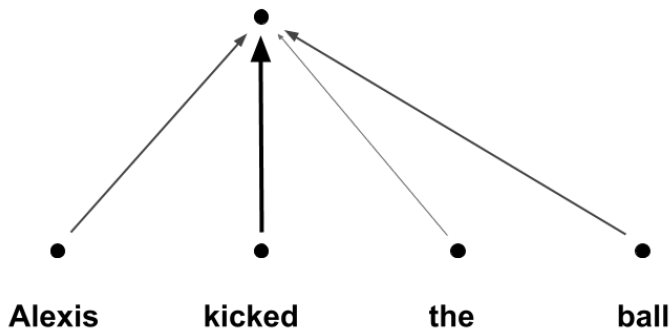
- Can be easily parallelizable.
- One step distance to every other position.
- Can be stacked forming a hierarchy.



Let's peek at the lab for a clearer picture.

Why do we have to restrict what a given word is focusing on?

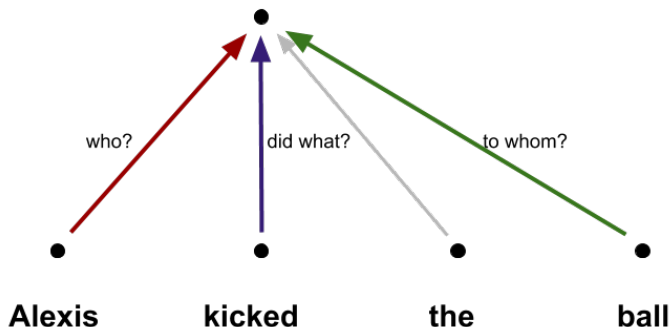
E.g. the verb embedding might want to focus on the subject as well as the object.



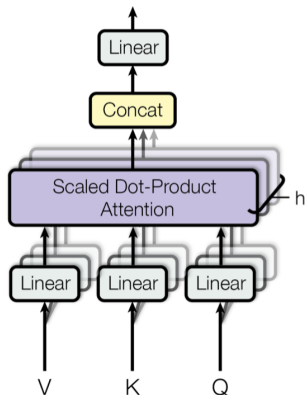
Multi-Head Attention

Why do we have to restrict what a given word is focusing on?

E.g. the verb embedding might want to focus on the subject as well as the object.



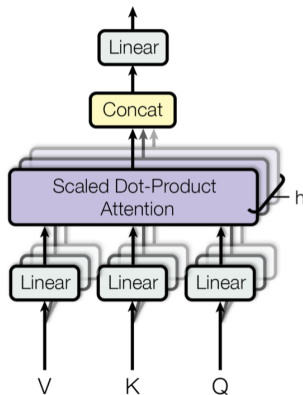
$$MultiHeadAttn(Q, K, V) = W^O Concat(head_1, \dots, head_h)$$
$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$$



Multi-Head Attention

$$\text{MultiHeadAttn}(Q, K, V) = W^O \text{Concat}(\text{head}_1, \dots, \text{head}_h)$$

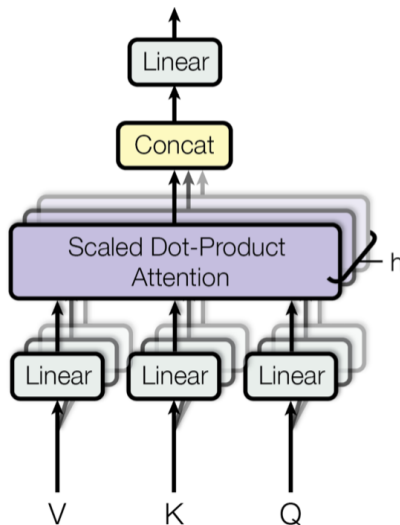
$$\text{head}_i = \text{Attention}(Q W_i^Q, K W_i^K, V W_i^V)$$



Multi-Head Attention

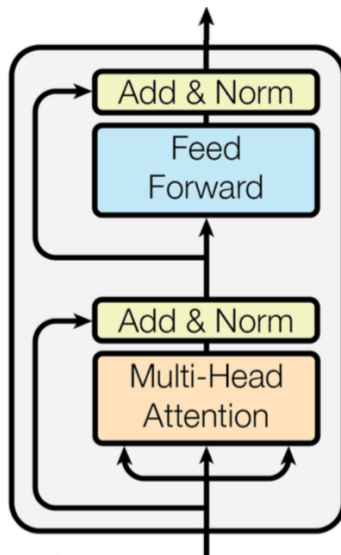
Allow the model to have multiple attention distributions per embedding.

Compose them to create new representations.

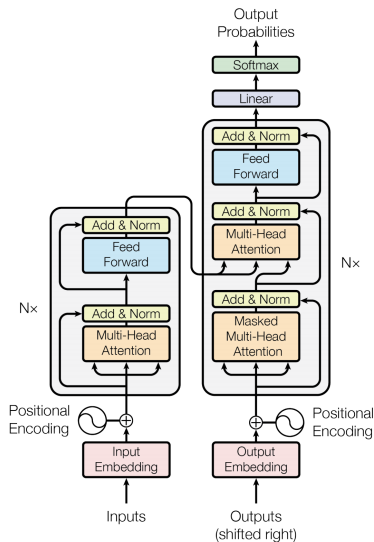


MultiHead Attention +

- Residual connections.
- Layer normalization
- Position-wise Feed-forward Layer ($\sim$ Fully connected layer).



Stack of N Transformer layers



GPT Comparison

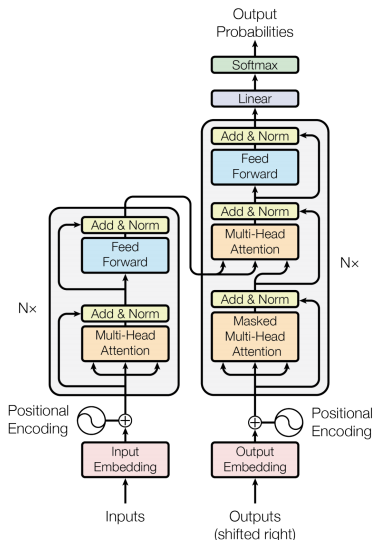
	Year	Inner Dimension	Number of Heads	Number of Layers	Parameters	Training Data
GPT	2018	768	12	12	0.12 Billion	BookCorpus: 4.5 GB of text.
GPT-2	2019	1,600	48	48	1.5 Billion	WebText: 40 GB of text
GPT-3	2020	12,288	96	96	175 Billion	CommonCrawl, WebText, English Wikipedia, Books 1 and 2: 570 GB of text.
GPT-4	2023	?	?	?	~1.76 Trillion	?

Positional embeddings

$$PE(p, 2i) = \sin(p/10000^{2i/d_{model}})$$

$$PE(p, 2i+1) = \cos(p/10000^{2i/d_{model}})$$

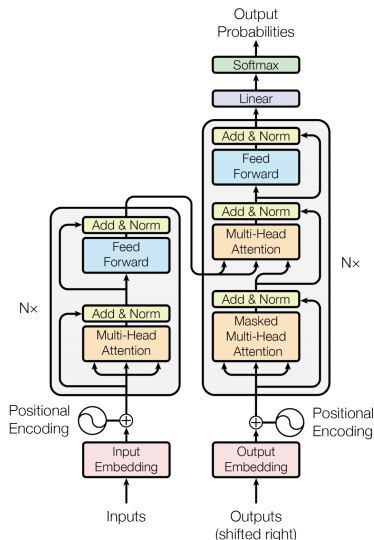
Or learned.



Encoder + Decoder

Decoder

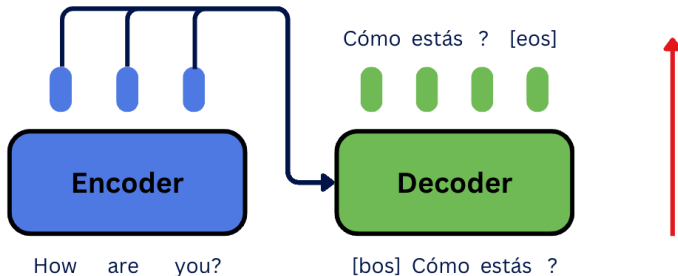
- Similar to an encoder with some considerations.
- Cross-attention.
- Masking of the future.



Encoder + Decoder

Decoder

- Similar to an encoder with some considerations.
- Cross-attention.
- Masking of the future.



Cross-Attention

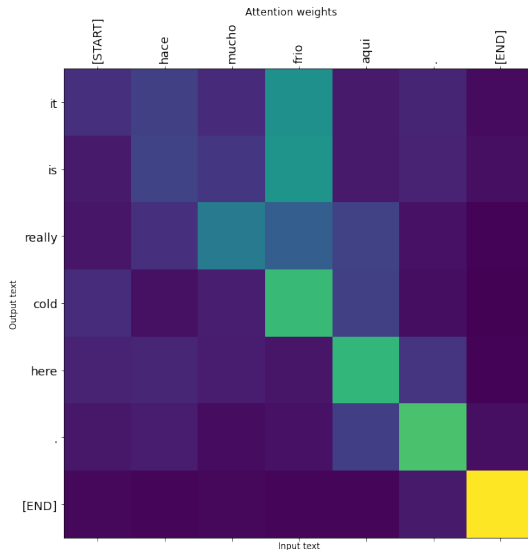




Figure: Click me!

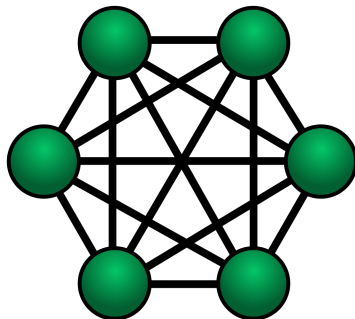
Transformer for Multi-modal Learning

Vision Transformers + CLIP

- 1 **Vision Transformers:** Model
- 2 **CLIP:** Training Scheme

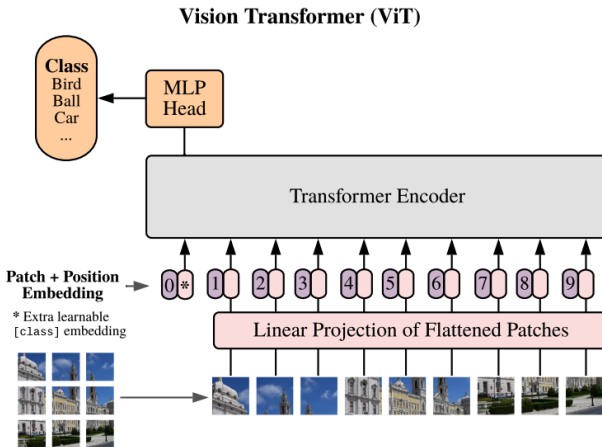
Issues with the transformer for images?

- Pixel space is much larger.
- Complexity scales O^2



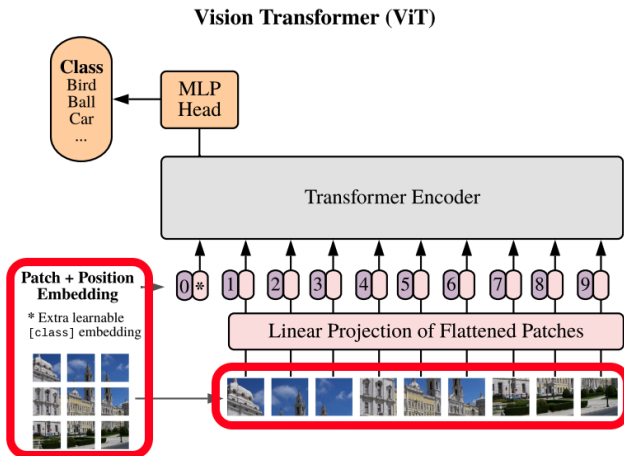
Vision Transformer Model

Overview



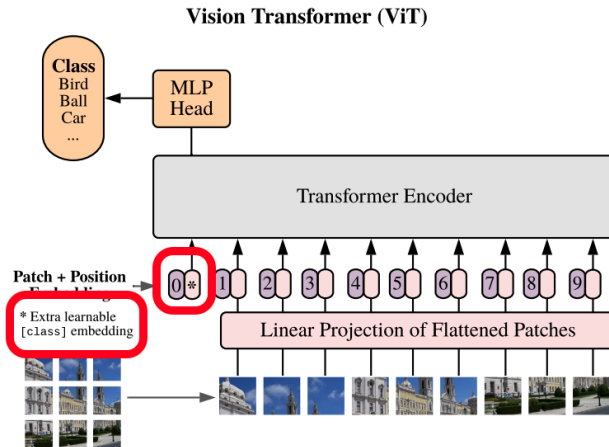
Vision Transformer Model

Divide the image in patches (originally 16x16)



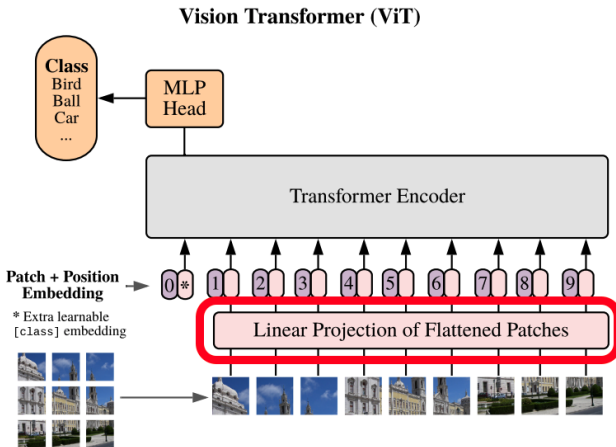
Vision Transformer Model

Add a special token for classification ([class]).



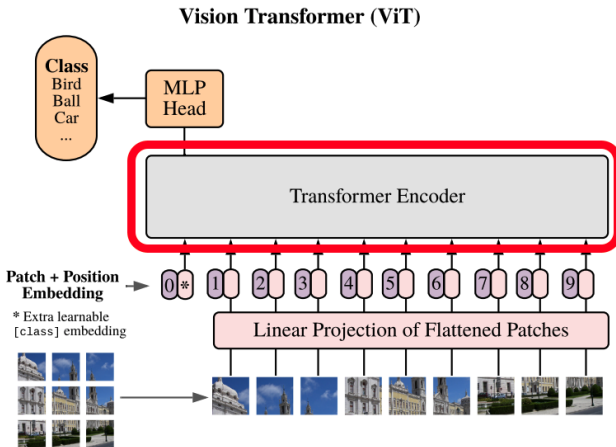
Vision Transformer Model

Linearly project to embedding space.



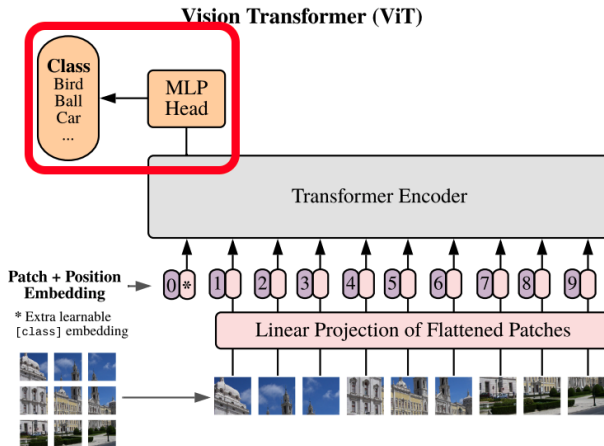
Vision Transformer Model

Based on the Transformer encoder.



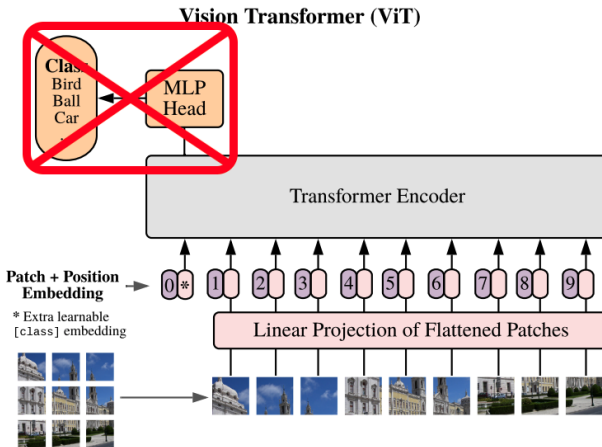
Vision Transformer Model

Classify using the special [class] token.



Vision Transformer Model

In clip, we will not use it for classification.



CNNs

- Locality
- Two-dimensional neighborhood structure
- Translation equivariance

vs

ViT

- Two-dimensional neighborhood structure: cutting the image into patches
- MLP layers: local and translationally equivariant.
- Self-attention layers are global.

How do we train it?

Vision datasets:

- Are labor intensive and costly to create.
- Test only a narrow set of visual concepts.

Standard vision models:

- Are good at one task and one task only.
- Require significant effort to adapt to a new task.
- Perform well on benchmarks have disappointingly poor performance on stress tests.

Contrastive Language–Image Pre-training

Ideas:

- Train model using vast data from the web pairing images and text.
- Using text as a more flexible label to address other tasks.
- Perform a “zero-shot” task using natural language.
- Learn a representation that’s connected to language.

WebImageText (WIT) Dataset

- 400 million image-text pairs
- Collected from a variety of sources on the Internet
- Enhance by searching for image-text pairs from a set of queries.
- Build a query list with all words occurring at least 100 times in Wikipedia.
- Including up to 20,000 image-text pairs per query.

Given a batch of N image-text pairs, CLIP is trained to predict which of the $N \times N$ possible image-text pairings across a batch occurred.

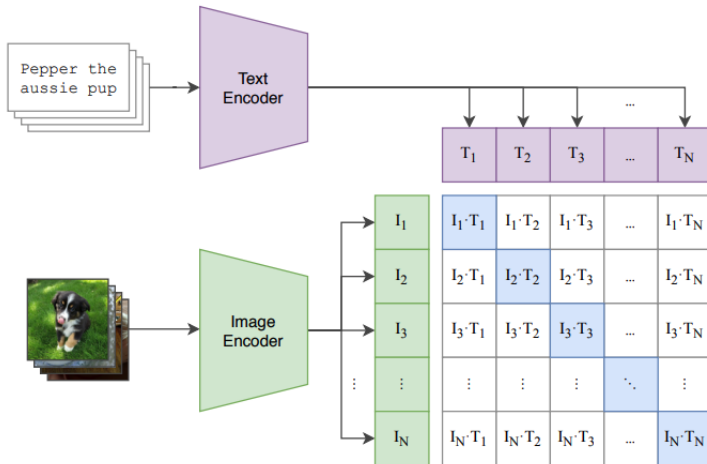
Contrastive objective:

- Related items should be closer together in the embedding space than unrelated ones.

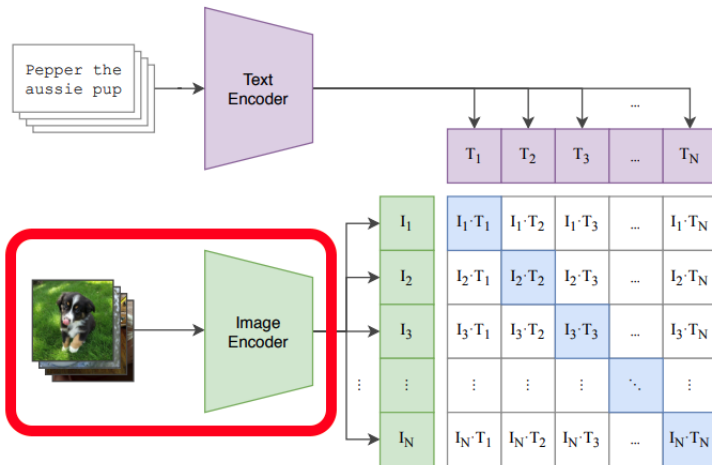
Why contrastive?

- Avoid trying to predict the exact words.
- More efficient.

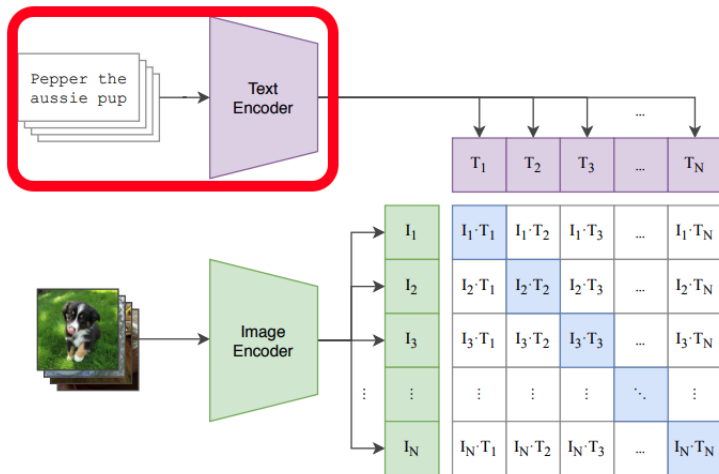
Overview.



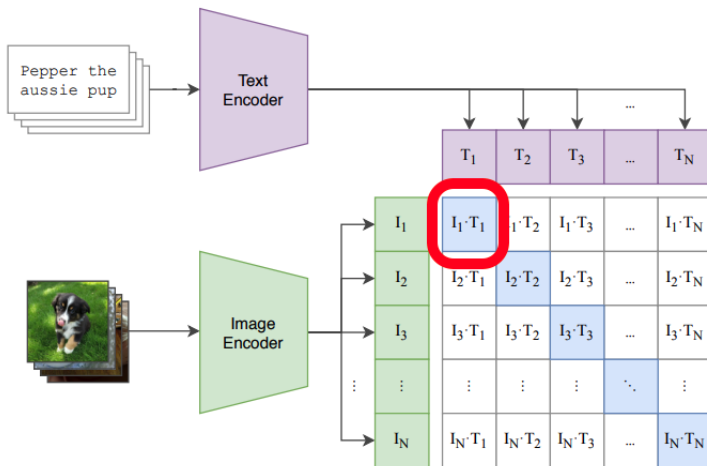
Encode image using Vision Transformer (ViT).



Encode text using a Transformer encoder.

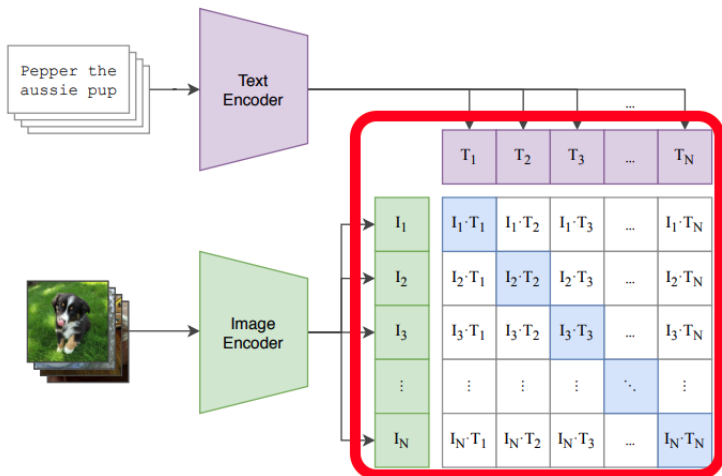


Cosine similarity between the embeddings.



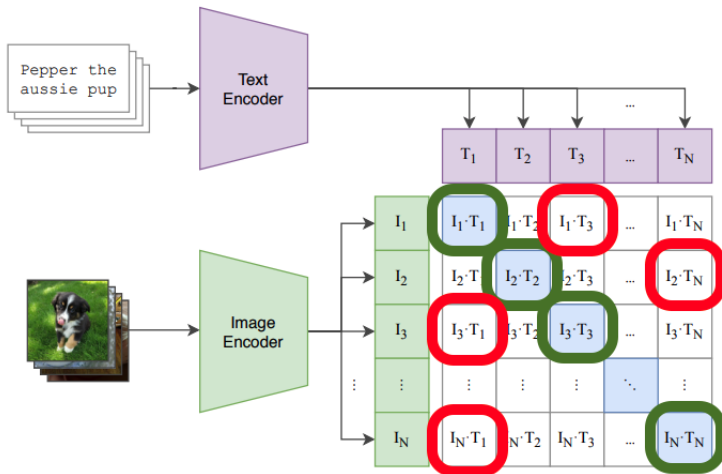
CLIP: Training

Use the whole batch for positive and negative examples.



CLIP: Training

Using cross-entropy. Positive examples get closer, and negative ones further apart.



Clips compare a text and an image, giving a similarity score.

Zero-shot Image Classification

- Classify using query “A photo of a {class}”.

FOOD101

guacamole (90.1%) Ranked 1 out of 101 labels



✓ a photo of **guacamole**, a type of food.

✗ a photo of **ceviche**, a type of food.

✗ a photo of **edamame**, a type of food.

✗ a photo of **tuna tartare**, a type of food.

✗ a photo of **hummus**, a type of food.

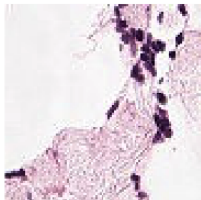
Clip compares a text and an image, giving a similarity score.

Zero-shot Image Classification

- Classify using query "A photo of a {class}".

PATCHCAMELYON (PCAM)

healthy lymph node tissue (22.8%) Ranked 2 out of 2



✗ this is a photo of **lymph node tumor tissue**

✓ this is a photo of **healthy lymph node tissue**

Final words

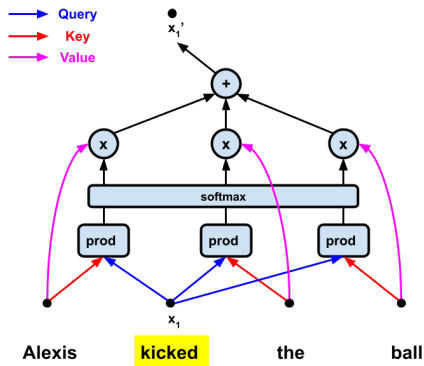
- BETO's repository.
- Harvard's Annotated Transformer.
- Stanford's NLP course session about Transformers.
- Original Transformer paper.
- Original BERT paper.
- *Analysis of BETO's attention patterns (Bertology).
- **Transformer Circuits.**

Questions?

Appendix

Why normalize?

When d_k grows, the dot product grows too pushing the softmax to small gradient regions.

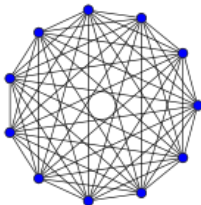


Self-Attention as a Relational Bias

We are computing relationships from all tokens (entities) with a given one.

Relational bias

- Tokens are the entities and attention models the relationship between them.
- Impose a fully connected graph.



Transformers for NLP: BERT

Replicate the success CNN has had in lots of tasks just by pre-training on Imagenet.

The model's architecture must support a large number of tasks.

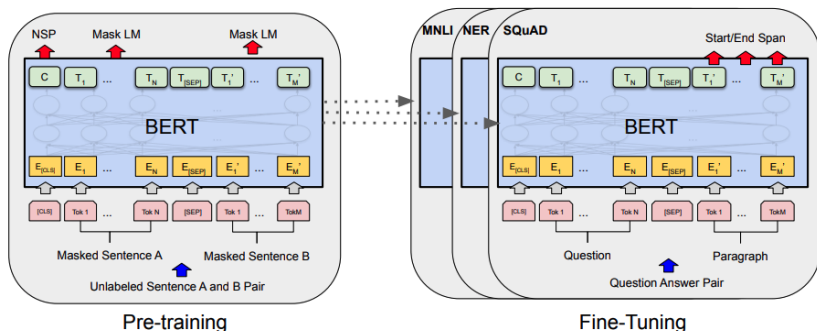
- Token classification tasks.
- Sentence classification tasks.
- Single sentence input.
- Pair of sentence input.

BERT

"BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding" by Devlin et al. 2018.

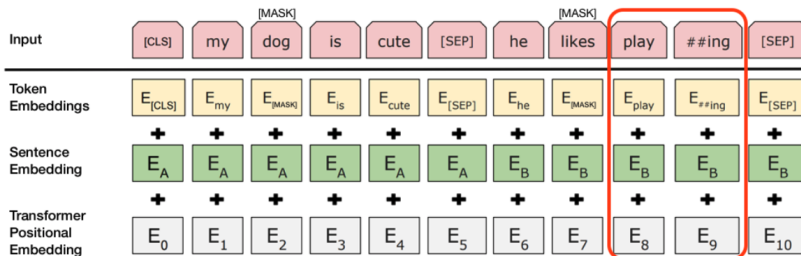
Multi-layer bidirectional Transformer encoder.

Unified architecture across tasks.



BERT Input Representation

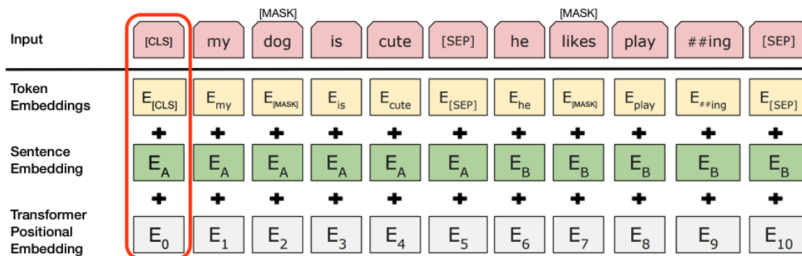
Word piece embeddings.



BERT Input Representation

First token: special classification token [CLS]

Used as the aggregate sequence representation for classification tasks.

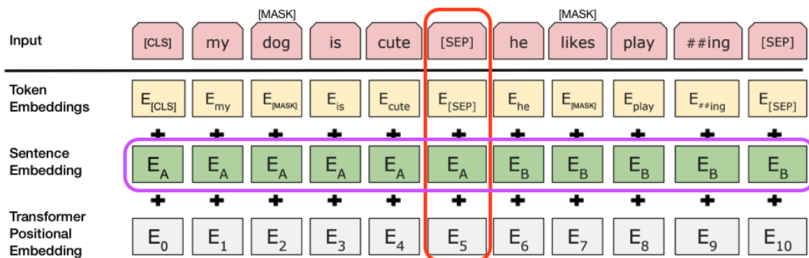


BERT Input Representation

Unambiguously represent single and pair of sentences.

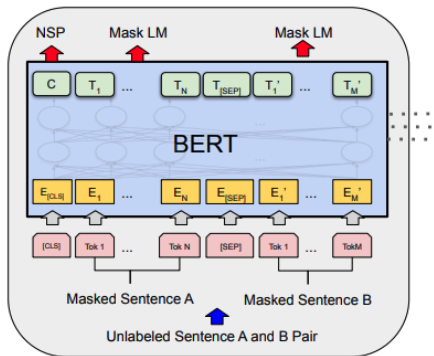
Differentiate the sentences in two ways.

- 1 Separate them with a special token, [SEP].
- 2 Add a learned embedding indicating which sentence it belongs to.



Masked language model.

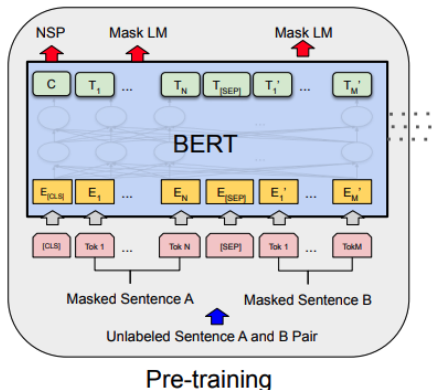
- Mask 15% of all tokens at random to predict that place token.
- Only 80% of the time use the [MASK] token.
- 10% with a random token.
- 10% without replacement.



Pre-training

Next sentence prediction.

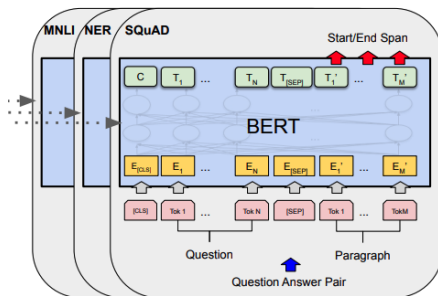
- Binary task of whether sentence B follows sentence A.
- 50% of times is and 50% isn't.
- Classified using the [CLS] special token.



Input

Sentences A and B are used as follows in the following tasks.

- Sentence pairs in paraphrasing.
- Hypothesis-premise pairs in entailment.
- Question-passage pairs in question answering.
- No text pair for text classification or sequence tagging.

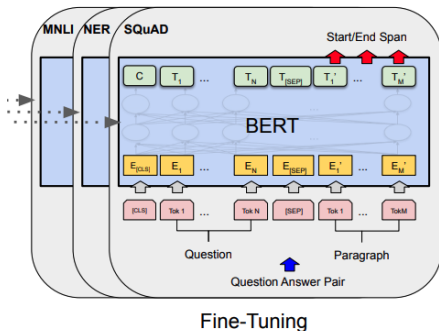


Fine-Tuning

Output

The token representations are fed into an output layer for token-level tasks, i.e. sequence tagging for question answering.

The [CLS] final representation is fed into an output layer for classification.



CLIP

Pseudo-código de Entrenamiento

```
# image_encoder - ResNet or Vision Transformer
# text_encoder  - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l]       - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t            - learned temperature parameter

# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T)  #[n, d_t]

# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)

# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss   = (loss_i + loss_t)/2
```

Relation Networks

A simple neural network module for relational reasoning

Adam Santoro*, David Raposo*, David G.T. Barrett, Mateusz Malinowski,
Razvan Pascanu, Peter Battaglia, Timothy Lillicrap

adamsantoro@, draposo@, barrettdavid@, mateuszm@,
razp@, peterbattaglia@, countzero@google.com

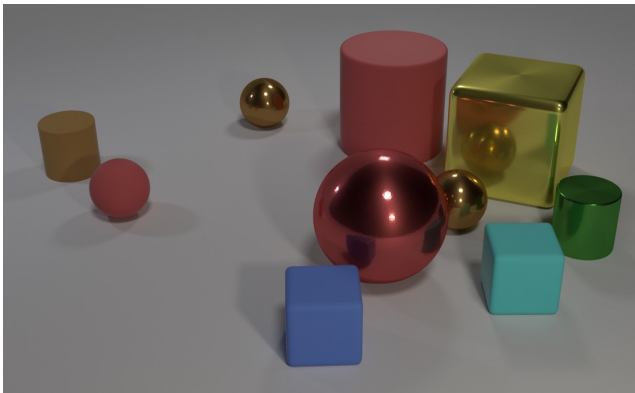
DeepMind
London, United Kingdom

"RNs are simple, plug-and-play, and are exclusively focused on flexible relational reasoning".

$$RN(O) = f_{\phi} \left(\sum_{i,j} g_{\theta}(o_i, o_j) \right)$$

Learn functions g_{θ} and f_{ϕ}

- g_{θ} encodes the relationship between the entities $o_i, o_j \in O$.
- f_{ϕ} aggregates all relationships.



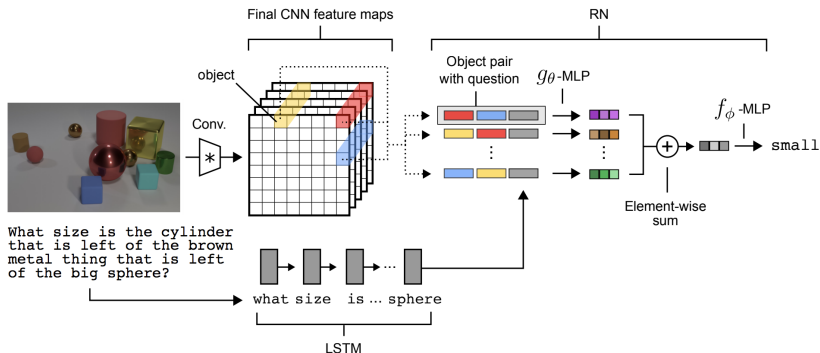
What size is the cylinder that is left of the brown metal thing that is left of the big sphere?

Answer: small

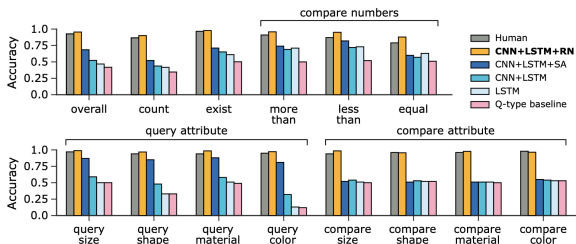
Model for the CLEVR Task

Pixel input & question embedding

$$RN(O) = f_{\phi} \left(\sum_{i,j} g_{\theta}(o_i, o_j, \mathbf{q}) \right)$$



Results on the CLEVR task.



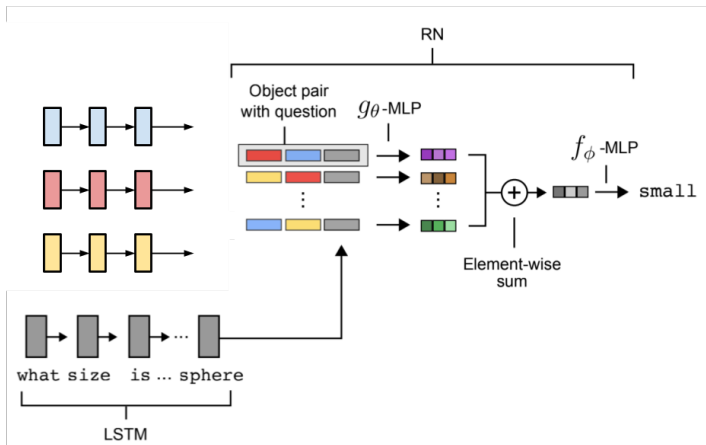
- 1 Mary moved to the bathroom.
- 2 John went to the hallway.
- 3 Where is Mary? bathroom 1
- 4 Daniel went back to the hallway.
- 5 Sandra moved to the garden.
- 6 Where is Daniel? hallway 4
- 7 John moved to the office.
- 8 Sandra journeyed to the bathroom.
- 9 Where is Daniel? hallway 4
- 10 Mary moved to the hallway.
- 11 Daniel traveled to the office.
- 12 Where is Daniel? office 11
- 13 John went back to the garden.
- 14 John moved to the bedroom.
- 15 Where is Sandra? bathroom 8

Model for bAbi Task

Each sentence is considered an object.

They have their position tagged.

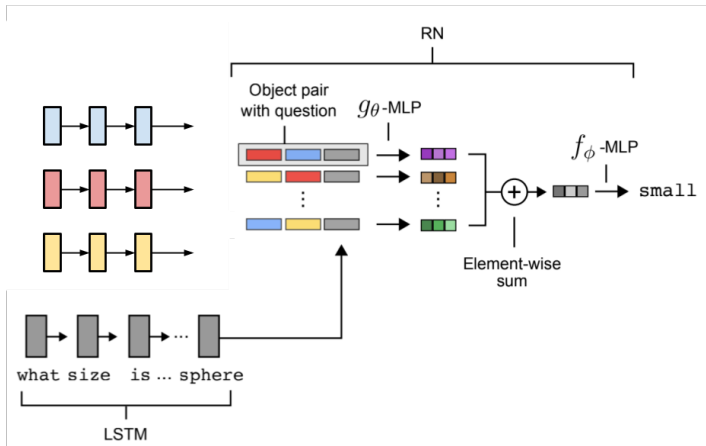
The input to the RN module is the last hidden state of the encoding LSTM.



Model for bAbi Task

Results. Passed 18/20 tasks.

Other such as Memory Networks pass 14/20, DNC 18/20, Sparse DNC 19/20, and EntNet 16/20.



- Flexible.
- Learn to infer relationships.
- Data efficient.
- Invariant to object order (operates over the object set).